AES-128 CRYPTOGRAPHY LABORATORY

Interactive Dockerized Teaching and Analysis Tool

TECHNICAL PROJECT REPORT

Project Domain	Cryptography / Cybersecurity Education
Implementation	Python Tkinter + FastAPI + Docker
Cryptographic Scope	AES-128
Report Focus	Implementation Logic, Workflow, Validation and Results
Date	October 2026

Prepared as a concise technical record of the completed implementation.

Abstract

This project implements an interactive AES-128 cryptography laboratory intended for teaching, visualization and experimental analysis. The system combines a native Python Tkinter graphical interface with a Dockerized FastAPI backend. A custom AES-128 teaching engine executes encryption while capturing the internal state after each transformation, allowing the user to inspect 41 states spanning the input, initial AddRoundKey, nine full rounds and the final round. Dedicated laboratories expose SubBytes, ShiftRows, MixColumns, AddRoundKey and AES-128 key expansion at byte and matrix level. An avalanche-analysis subsystem performs controlled single-bit changes to plaintext or key data, measures Hamming distance across AES states, plots diffusion, and supports multi-trial statistical experiments. The student experiment mode produces observation tables, graphical analysis, CSV datasets and a formal text lab record. The implementation was validated using the standard AES-128 known-answer vector, producing ciphertext 69C4E0D86A7B0430D8CDB78070B4C55A. The completed system provides a reproducible, containerized and interactive platform for examining AES-128 implementation behaviour without requiring students to manually trace every round.

1. Introduction

AES is commonly introduced through formulas, fixed examples and final input/output pairs. That approach does not readily expose how the 4 x 4 byte state evolves, how round keys are derived, or how a one-bit input change propagates through successive transformations. The project therefore focuses on implementation transparency: the software makes internal AES-128 states observable and connects those states to dedicated transformation and experiment views.

The project is intentionally restricted to AES-128. It is not a general cryptography suite. This constraint allows the interface, backend and experiment logic to remain detailed and internally consistent while supporting both guided exploration and repeatable laboratory work.

1.1 Project Objectives

- Provide an interactive view of all captured AES-128 encryption states.
- Separate the four AES round transformations into dedicated, inspectable laboratories.
- Expose the AES-128 key schedule from W0 to W43 and all 11 round keys.
- Measure plaintext-bit and key-bit avalanche behaviour using Hamming distance.
- Support repeatable student experiments with tables, graphs, statistics and exportable results.
- Package the cryptographic backend in Docker and expose it through a REST API.

2. System Scope and Functional Requirements

The final application is an AES-128 educational simulator rather than a production encryption product. The central requirement is that every visualization be derived from the actual backend computation rather than from static demonstration values. The same input plaintext and key therefore drive the step visualizer, transformation laboratories, key schedule and avalanche analysis.

2.1 Final Functional Scope

Module	Primary Function	Key Output
Step-by-Step Visualizer	Navigate the instrumented AES execution	41 internal states, round key when applicable
Transformation Laboratory	Compare state immediately before and after each transformation	40 transformation transitions
SubBytes / S-Box	Trace byte substitution through the 16 x 16 forward S-box	Nibble lookup and substituted byte
ShiftRows	Trace cyclic row movement	Source and destination positions
MixColumns / GF(2^8)	Explain finite-field column mixing	Per-byte GF calculation and

		backend match
AddRoundKey / XOR	Inspect byte-wise state/key XOR	Hex and binary XOR verification
Key Expansion	Inspect AES-128 key schedule	W0-W43 and round keys 0-10
Avalanche Laboratory	Measure diffusion for one flipped plaintext/key bit	41-step Hamming-distance graph
Student Lab Mode	Run multi-trial avalanche experiments	Table, statistics, graph, CSV and report

The GUI also contains a compact AES-128 reference panel. A practical AES-GCM utility remains part of the backend history, but the final dashboard emphasizes the raw AES-128 teaching and analysis workflow so that authenticated encryption utility logic is not confused with the internal round visualizer.

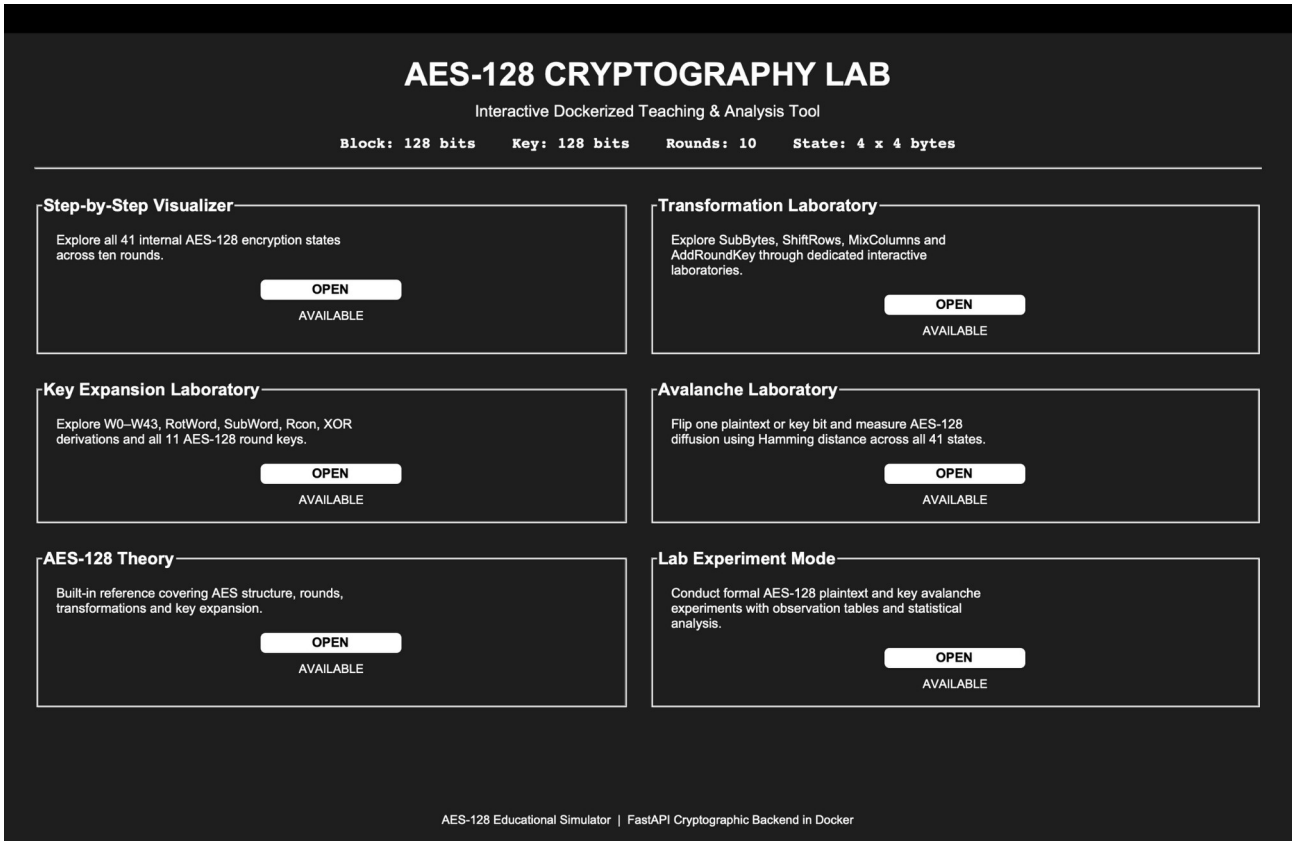


Figure 1. Main AES-128 Cryptography Laboratory dashboard.

3. System Architecture and Technology Stack

The application uses a two-tier local architecture. The Tkinter GUI runs natively on the host computer and handles interaction, presentation and embedded Matplotlib charts. Computational requests are sent as JSON over HTTP to a FastAPI service running inside Docker. The backend validates inputs, executes the AES teaching engine or experiment logic, and returns structured JSON. This separation keeps the teaching interface independent of the cryptographic execution environment and makes the backend reproducible across systems with Docker support.

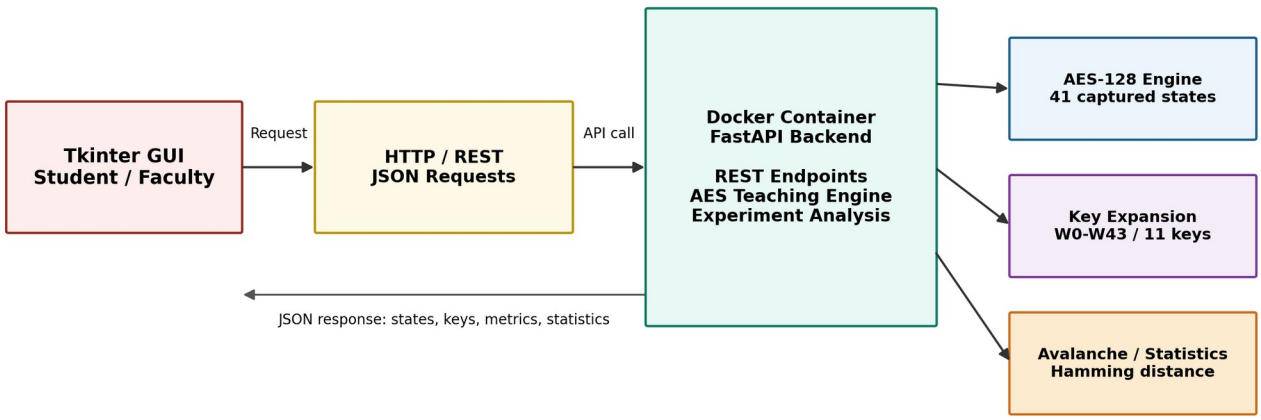


Figure 2. Overall system architecture and request/response path.

3.1 Technology Stack

Technology	Role
Python 3	Core implementation language
Tkinter / ttk	Desktop graphical interface and tables
FastAPI	REST service and input/output models
Docker	Isolated backend runtime and dependency packaging
Matplotlib	Avalanche propagation and experiment graphs
cryptography	Practical AES-GCM utility path
csv / statistics / standard library	Export and statistical support

3.2 API-Oriented Data Flow

1. The GUI collects a 16-byte plaintext and 16-byte AES-128 key in hexadecimal form.
2. The selected module creates a JSON request and sends it to the relevant FastAPI endpoint.
3. The backend executes the instrumented AES engine, key expansion routine, or avalanche experiment.
4. Structured results are returned as JSON rather than pre-rendered text.
5. The GUI converts the returned matrices, keys, observations and statistics into interactive views.

4. Application Workflow and Module Navigation

The main dashboard is the control point for the final application. Each laboratory is independent at the presentation layer but consumes the same AES-128 backend representation. This avoids inconsistencies between a transformation shown in one module and the corresponding state shown elsewhere.

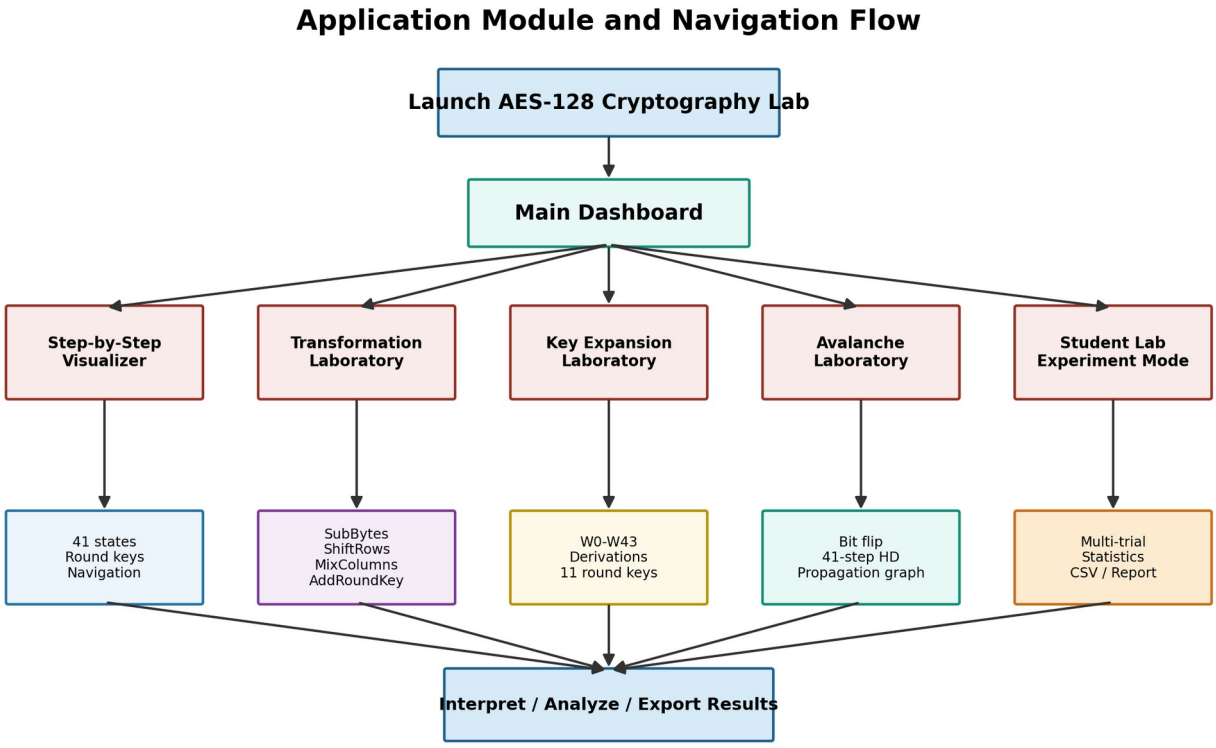


Figure 3. Application module and navigation workflow.

4.1 Common Input Convention

AES data is represented as 32 hexadecimal characters for both plaintext and key. Internally, the AES state is displayed in the standard column-major 4 x 4 byte arrangement. The known-answer plaintext 00112233445566778899AABBCCDDEEFF and key 000102030405060708090A0B0C0D0E0F are used throughout the screenshots and validation because they allow direct comparison with the published AES example.

4.2 Shared State Model

The backend records a step number, AES round, operation name, state matrix and description for each captured state. AddRoundKey steps additionally include the round key. The resulting list is the common data source for the step visualizer, before/after transformation explorer and per-round transformation laboratories.

5. Instrumented AES-128 Execution Logic

The teaching engine is a pure-Python AES-128 block implementation instrumented to record intermediate states. The execution begins with the input state, records the initial AddRoundKey, then records SubBytes, ShiftRows, MixColumns and AddRoundKey for rounds 1 through 9. Round 10 records SubBytes, ShiftRows and AddRoundKey only. This produces 41 captured states: one input state, one initial AddRoundKey state, 36 states across rounds 1-9, and three states in the final round.

Stage	Operations Recorded	Count	Cumulative States
Input	Input state	1	1
Round 0	AddRoundKey	1	2
Rounds 1-9	SubBytes, ShiftRows, MixColumns, AddRoundKey	36	38
Round 10	SubBytes, ShiftRows, AddRoundKey	3	41

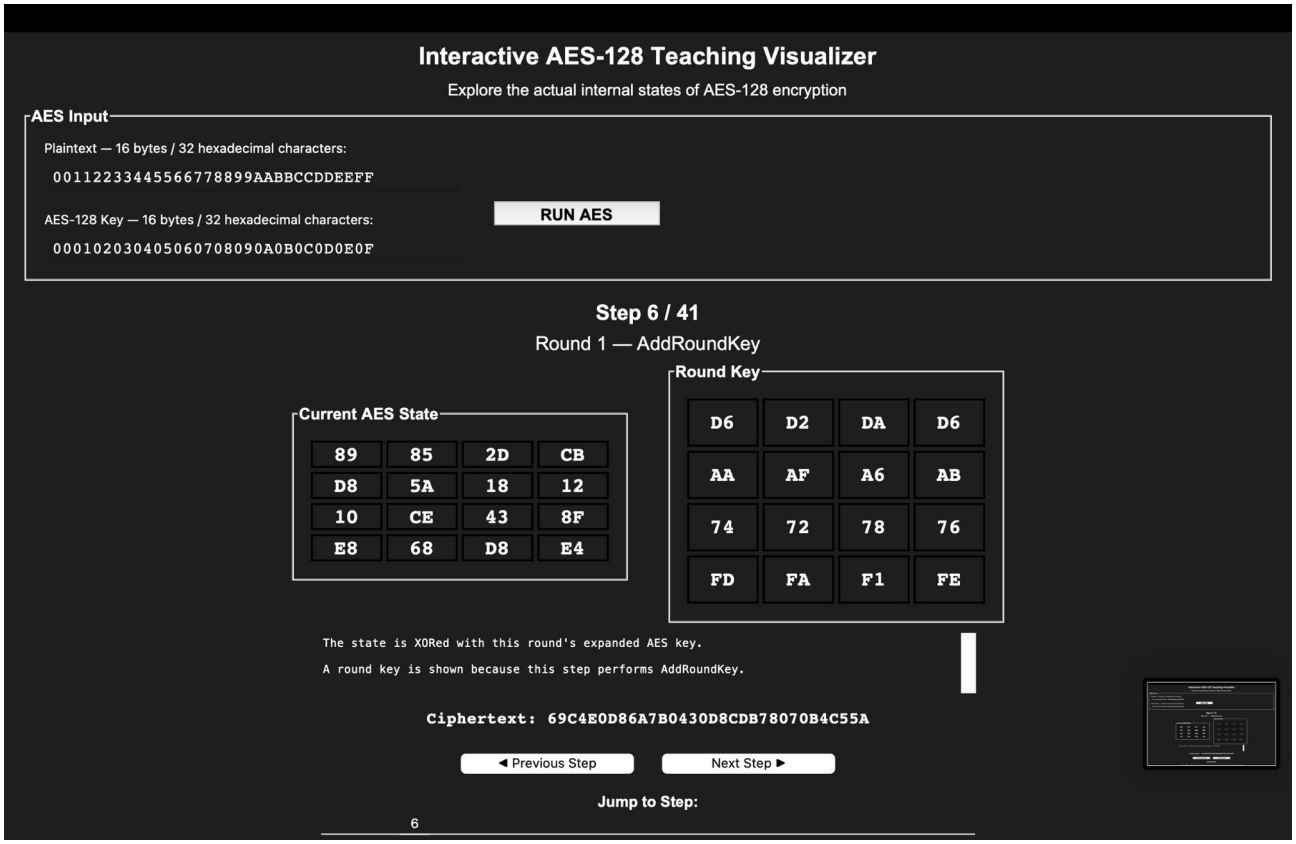


Figure 4. Step-by-step visualizer at an AddRoundKey state; the active round key is shown beside the AES state.

5.1 Navigation Logic

The GUI stores the returned step list and renders one indexed state at a time. Previous/Next controls and a 1-41 slider change only the selected index; AES is not recomputed for every navigation action. This makes exploration responsive and guarantees that the displayed ciphertext and intermediate states belong to the same backend execution.

6. Transformation Laboratory Implementation

The general transformation explorer treats each state transition after the input as one transformation. With 41 captured states, there are 40 transitions. For transition i, the state at index i-1 is the BEFORE matrix and the state at index i is the AFTER matrix. The operation label is taken from the current step. This simple indexing rule allows the interface to inspect every transformation without duplicating AES logic in the GUI.

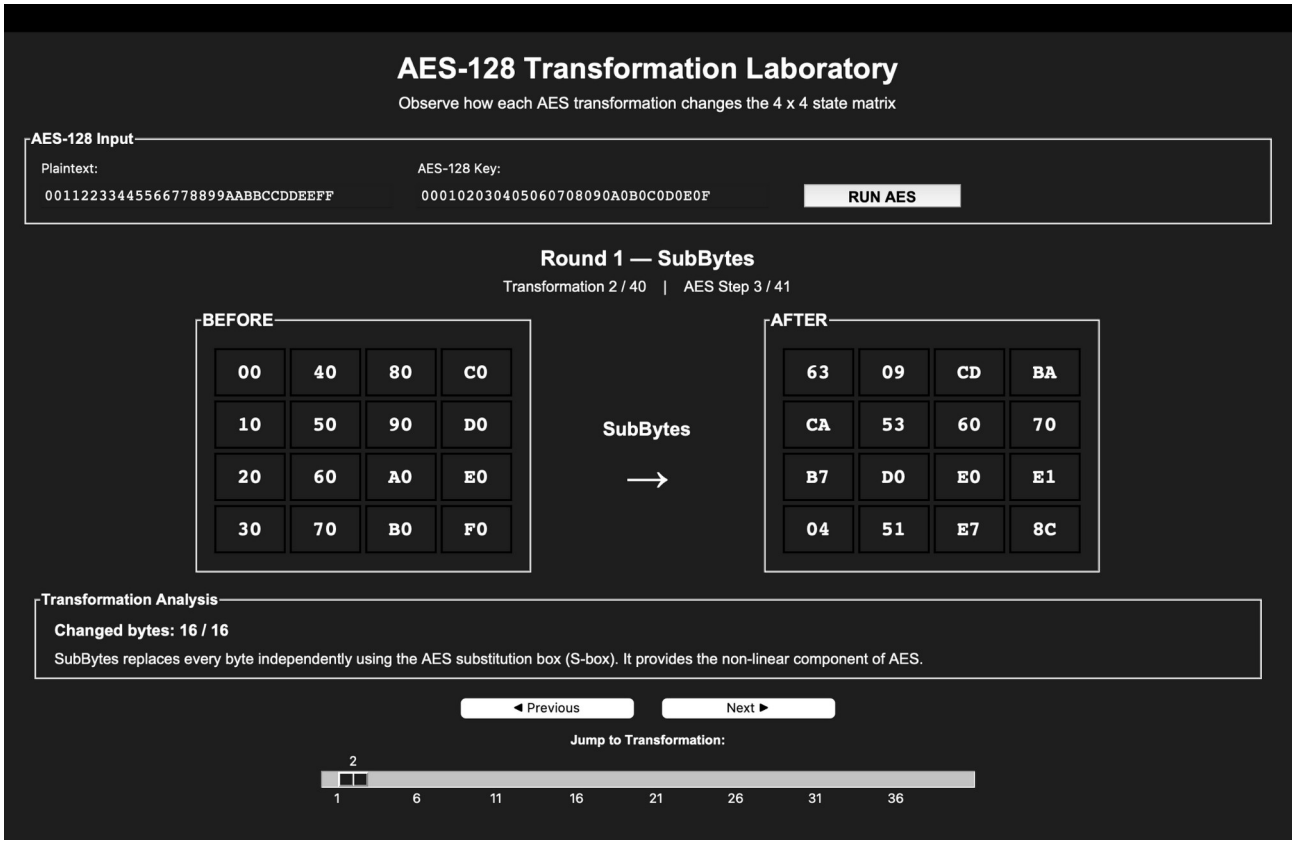


Figure 5. Transformation Laboratory showing a real SubBytes before/after transition.

6.1 Dedicated Transformation Views

The transformation hub provides deeper views for each operation. SubBytes presents the complete 16 x 16 forward S-box and maps the selected byte using its high and low nibbles. ShiftRows preserves byte values while tracing their changed positions. MixColumns independently calculates finite-field products and XOR combinations for a selected output byte, then checks the result against the backend. AddRoundKey selects the same byte coordinate in state, round key and output and displays both hexadecimal and 8-bit binary XOR.

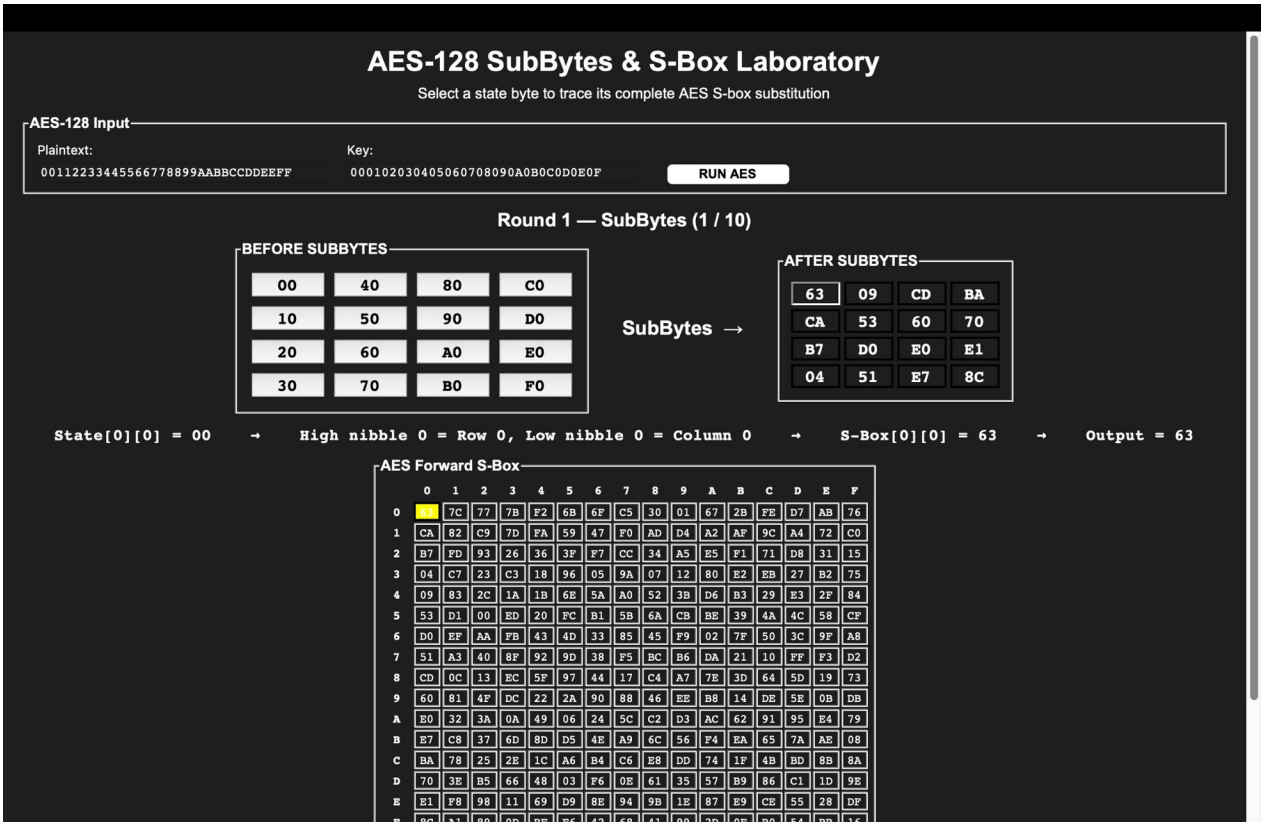


Figure 6. SubBytes and S-Box laboratory with the selected lookup highlighted.

7. Transformation Detail: ShiftRows, MixColumns and AddRoundKey

ShiftRows uses the destination rule (column - row) mod 4 for the displayed source byte. The four rows are shifted left by 0, 1, 2 and 3 positions respectively. Because the transformation changes position rather than value, the interface reports both original and destination coordinates.

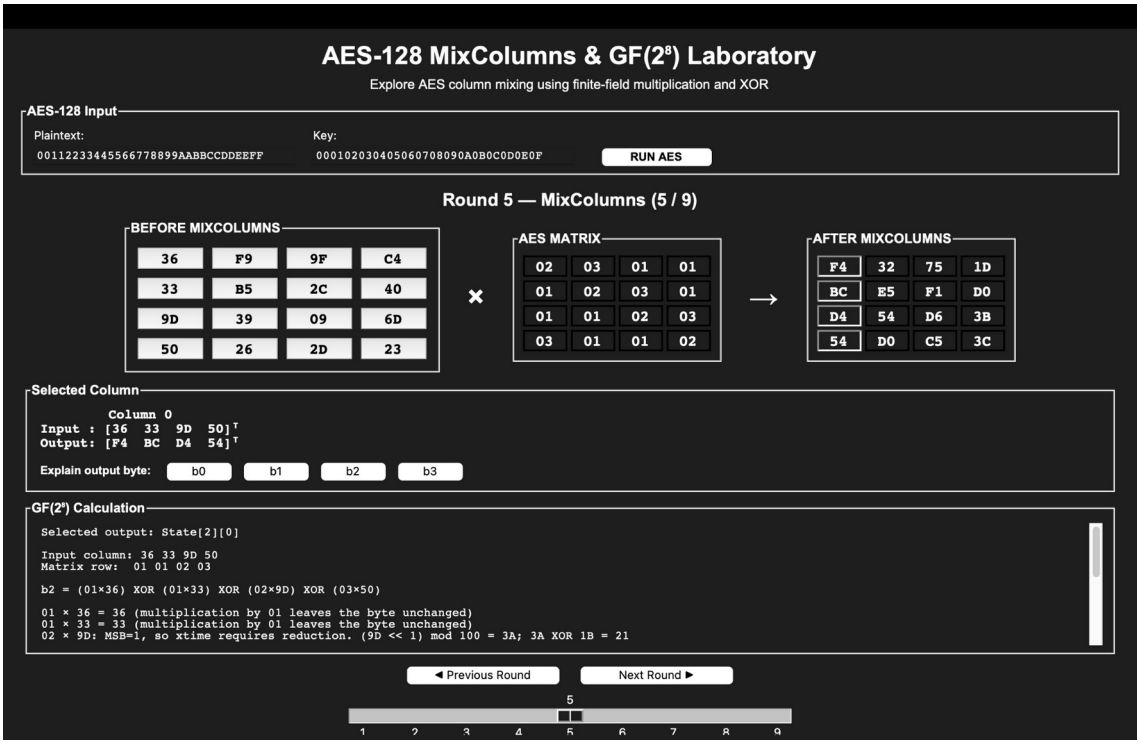


Figure 8. MixColumns laboratory with the fixed AES matrix and GF(2^8) calculation area.

MixColumns is available in rounds 1-9 only. The implementation uses multiplication by 01, 02 and 03 in GF(2^8), with reduction based on the AES polynomial and 0x1B reduction constant. For each selected column and output byte, the GUI reconstructs the equation and verifies that the independently calculated result matches the backend state.

8. Key Expansion Laboratory

AES-128 key expansion starts with four 32-bit words W0-W3 taken directly from the 128-bit key and derives W4-W43. Every fourth word applies the key-schedule core: RotWord, SubWord and Rcon, followed by XOR with the word four positions earlier. Other words are derived by XOR with the previous word and the word four positions earlier. The backend returns both the final words and derivation metadata so the GUI can explain how a selected word was produced.

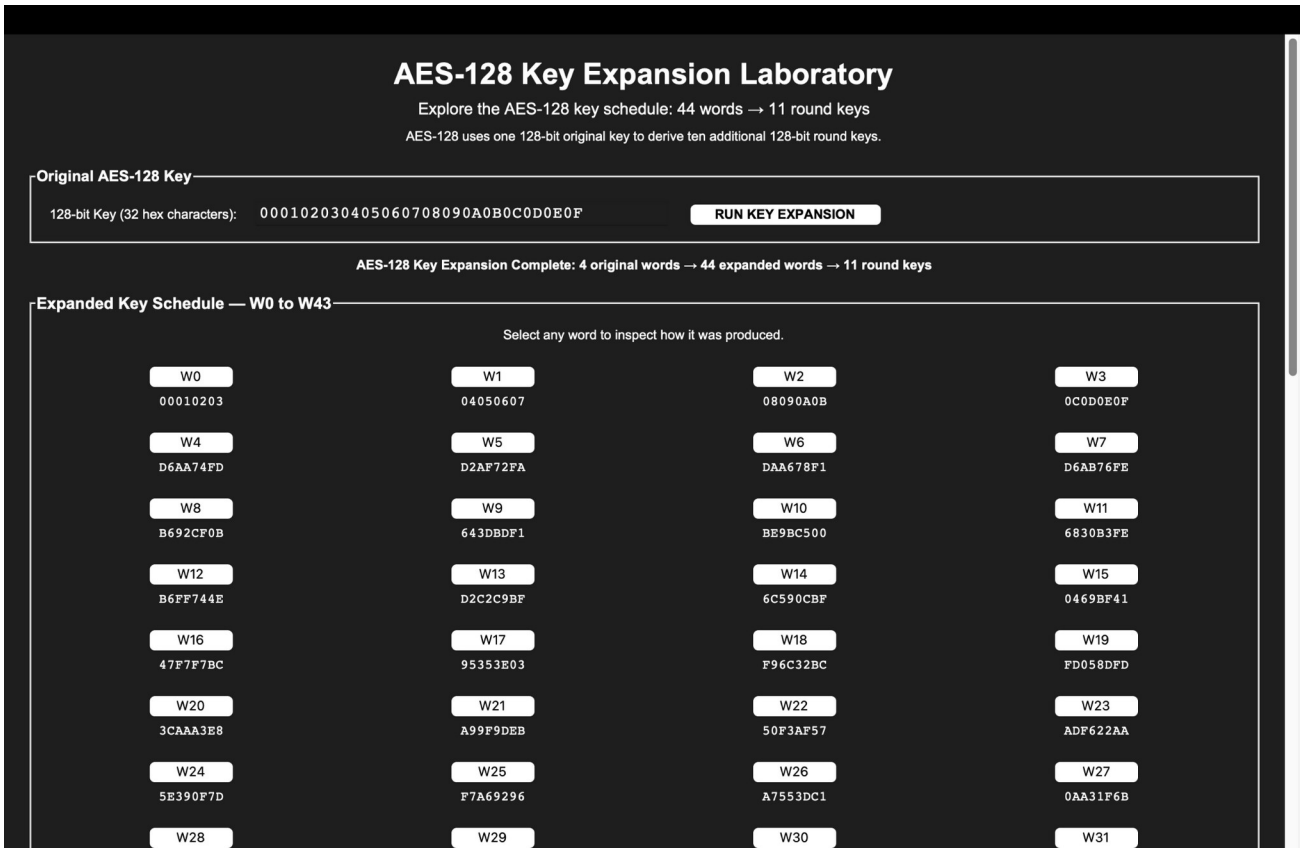


Figure 9. Key Expansion Laboratory displaying the W0-W43 schedule.

8.1 Round-Key Assembly

Every four consecutive expanded words form one 128-bit round key, producing round keys 0 through 10. For the validation key, the first derived words are W4=D6AA74FD, W5=D2AF72FA, W6=DAA678F1 and W7=D6AB76FE. The final round key is 13111D7FE3944A17F307A78B4D2B30C5. These values are used by the instrumented AES execution and can be inspected independently in the key expansion laboratory.

8.2 AddRoundKey Integration

There are 11 AddRoundKey transformations in AES-128: the initial operation in round 0 and one in each of rounds 1-10. The dedicated XOR laboratory displays the state before the operation, the selected round key, and the state after the operation, making the connection between key expansion and round execution explicit.

9. Avalanche Analysis Implementation

The avalanche subsystem measures how a controlled one-bit modification propagates through AES-128. Bit index 0 is defined as the most significant bit of byte 0, while bit 127 is the least significant bit of byte 15. The user can choose either plaintext-bit or key-bit modification. The backend runs the original and modified encryptions, aligns their 41 captured states, converts each state to bytes, and computes Hamming distance by counting differing bits.

For a 128-bit state, avalanche percentage is calculated as (Hamming distance / 128) x 100. A 64-bit difference is displayed as the 50% reference line; individual trials are not expected to equal 64 exactly, and per-step diffusion is not required to increase monotonically.

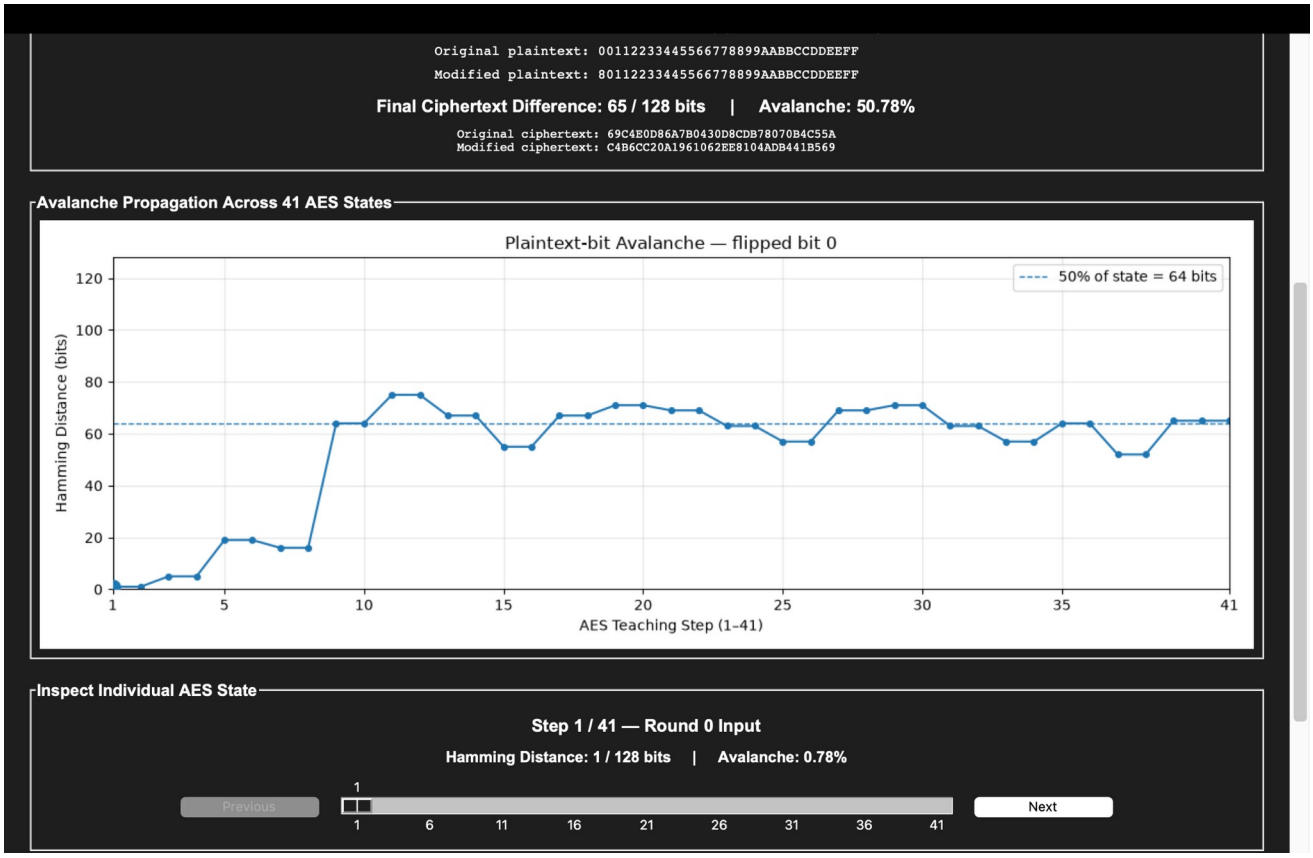


Figure 11. Avalanche propagation across all 41 AES teaching states after flipping plaintext bit 0.

9.1 Single-Bit Workflow

6. Create a modified plaintext or key by flipping exactly one selected bit.
7. Execute the instrumented AES-128 engine for original and modified inputs.
8. Compare corresponding states 1-41 and compute Hamming distance.
9. Return per-step distances, final ciphertexts and final avalanche percentage.
10. Plot propagation and allow the user to inspect the numerical result at any state.

10. Student Lab Experiment Mode

Student Lab Experiment Mode extends the single-bit analysis into a repeatable experiment. The user selects plaintext or key avalanche, enters the AES input values and chooses a bit range. Supported ranges contain 8, 16, 32, 64 or 128 trials. Each trial flips one distinct bit and records the final ciphertext Hamming distance and avalanche percentage.

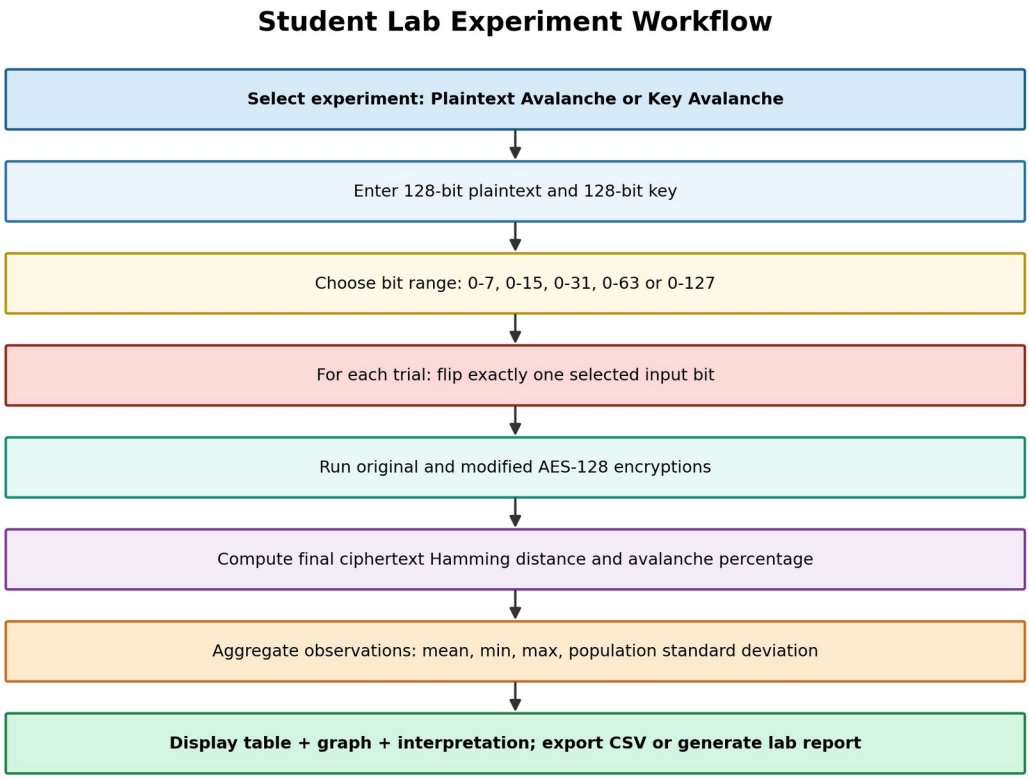


Figure 12. Multi-trial student experiment workflow.

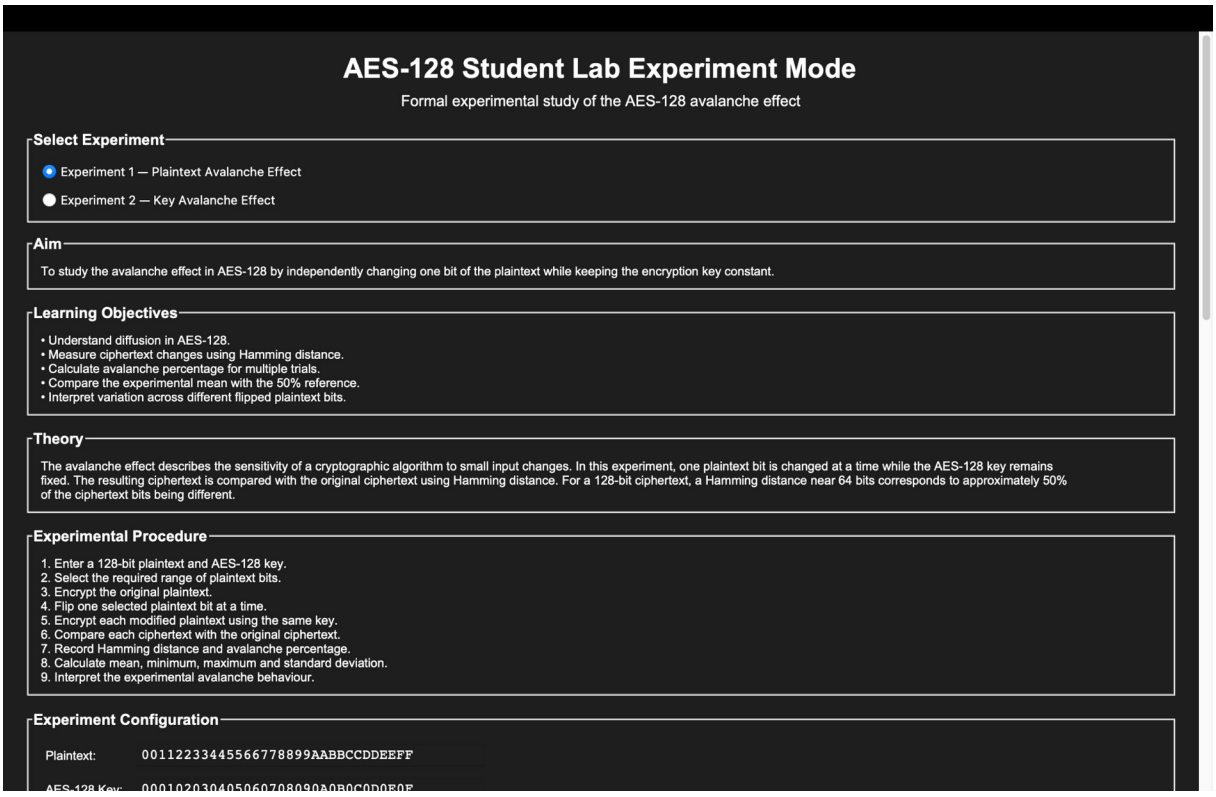


Figure 13. Student Lab Experiment Mode with aim, objectives, procedure and configuration.

10.1 Statistical Processing

After all trials are complete, the backend calculates count, mean, minimum, maximum and population standard deviation for Hamming distance and avalanche percentage. The GUI renders the observations in a Treeview table and plots final ciphertext Hamming distance against flipped-bit index. A dashed 64-bit line represents the 50% reference and a second reference line shows the experimental mean.

11. Results, Export and Report Generation



Figure 14. Eight-trial plaintext avalanche experiment observation table.

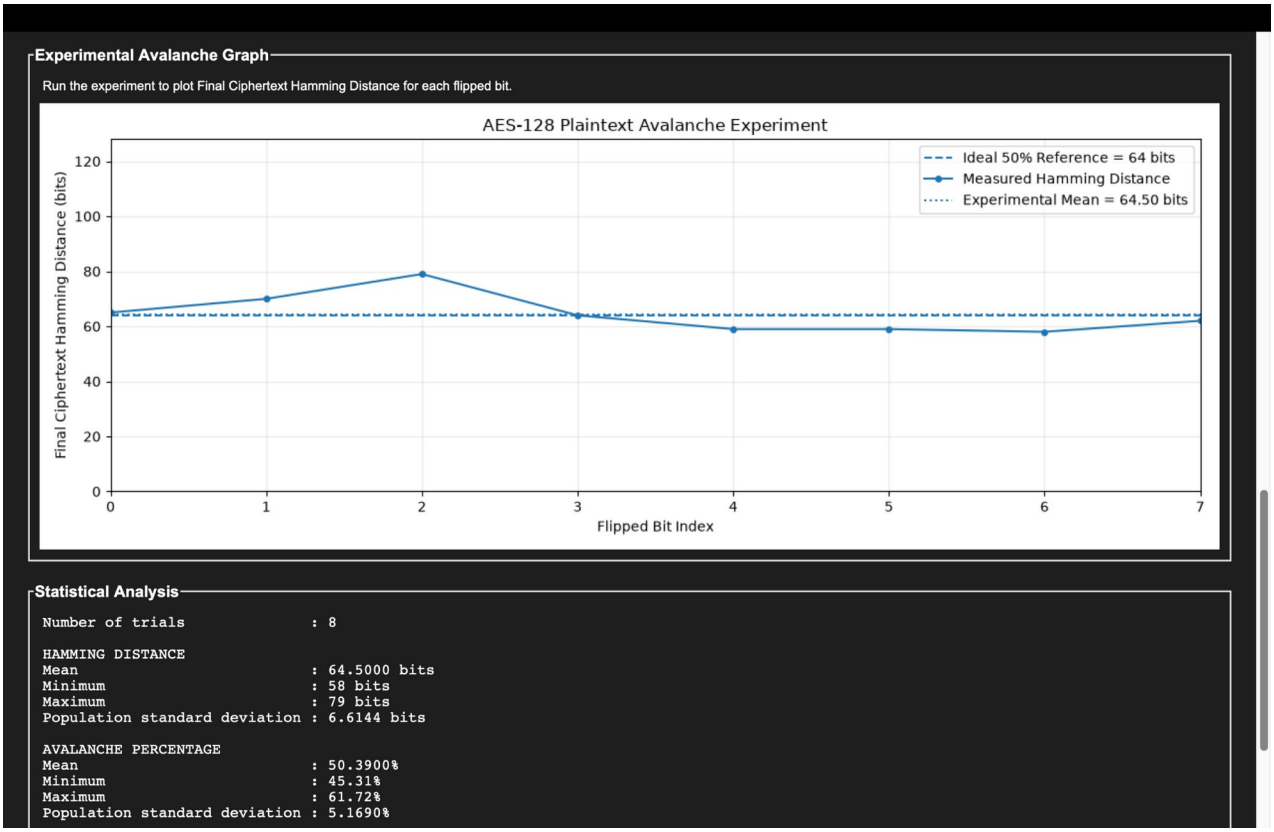


Figure 15. Experimental graph and statistical analysis for plaintext bits 0-7.

For the eight-trial plaintext experiment shown above, the measured Hamming distances are 65, 70, 79, 64, 59, 59, 58 and 62 bits. The mean is 64.5 bits, the minimum is 58 bits, the maximum is 79 bits and the population standard deviation is 6.6144 bits. The mean avalanche percentage is 50.3900%, close to the 50% reference for this specific experiment.

11.1 CSV Export

The CSV export stores experiment configuration, original plaintext and key, Hamming-distance statistics, percentage statistics, theoretical reference values, every trial observation, modified plaintext/key/ciphertext values and the backend interpretation. This provides a machine-readable dataset for later analysis in spreadsheet or statistical tools.

11.2 Formal Lab Record

The report generator converts the current experiment into an editable text-based laboratory record containing title, aim, learning objectives, experimental variables, configuration, procedure, observation table, statistical analysis, interpretation, conclusion and formulae. The report can be reviewed in the GUI before being saved.

12. Testing and Validation

Validation was performed at multiple levels: cryptographic known-answer testing, intermediate-state inspection, key-schedule checking, independent transformation calculations, API response verification and GUI-level experiment checks.

12.1 AES-128 Known-Answer Test

Plaintext	00112233445566778899AABBCCDDEEFF
Key	000102030405060708090A0B0C0D0E0F
Expected Ciphertext	69C4E0D86A7B0430D8CDB78070B4C55A

Application Ciphertext	69C4E0D86A7B0430D8CDB78070B4C55A

The application output matches the expected AES-128 ciphertext exactly. Intermediate checks also match the expected first-round states, including the Round 0 AddRoundKey state, Round 1 SubBytes state, ShiftRows state and MixColumns state.

12.2 Structural Validation

- 41 captured states for a complete AES-128 encryption.
- 40 transformation transitions between consecutive states.
- 10 SubBytes operations and 10 ShiftRows operations.
- 9 MixColumns operations because the final round omits MixColumns.
- 11 AddRoundKey operations including the initial round-0 operation.
- 44 key-schedule words W0-W43 and 11 round keys.
- Plaintext and key avalanche experiments over selectable ranges up to all 128 input bits.

12.3 Example Independent Checks

The MixColumns laboratory independently reconstructs GF(2^8) arithmetic and compares the result with the backend. The AddRoundKey laboratory independently computes selected byte XOR in hexadecimal and binary. These checks reduce the risk that the GUI merely repeats a backend value without validating the transformation logic.

13. Limitations and Future Enhancements

The project is deliberately narrow in cryptographic scope. It supports AES-128 teaching and analysis and does not attempt to compare AES with RSA, ECC, SHA or other algorithms. The teaching engine visualizes raw AES block operations; it should not be interpreted as a recommendation to use raw AES blocks for real-world data protection. The practical AES-GCM path exists separately because authenticated encryption has different operational requirements.

The GUI currently runs as a desktop Tkinter application while the backend runs locally in Docker. This design is appropriate for a laboratory environment but is not a multi-user web deployment. The macOS implementation also relies on a visible vertical scrollbar for long laboratory screens because native trackpad wheel events were not consistently emitted by the Tk 9 Aqua environment during testing.

13.1 Potential Enhancements

- Package the GUI and backend for simplified one-command distribution.
- Add PDF generation for the formal lab record while retaining CSV export.
- Add automated regression tests for all documented intermediate AES states and round keys.
- Provide optional experiment comparison views for larger trial sets without expanding beyond AES-128.
- Add accessibility and responsive-layout refinements for different desktop resolutions.

14. Conclusion

The completed AES-128 Cryptography Laboratory integrates an instrumented AES implementation, a Dockerized REST backend and an interactive desktop interface into one teaching and analysis system. Its main contribution is not another encryption utility, but an observable execution model: students can move from a 41-state overview to individual transformations, inspect the key schedule, introduce controlled bit changes, measure diffusion, and convert those measurements into repeatable laboratory observations. The standard AES-128 test vector and multiple independent transformation checks demonstrate that the visualized states are derived from the implemented algorithm rather than from static examples. The final system therefore meets the project goal of providing a detailed, reproducible and experiment-oriented AES-128 learning environment.

References

[1] National Institute of Standards and Technology (NIST), FIPS 197: Advanced Encryption Standard (AES), updated edition, 2023.

[2] NIST, Advanced Encryption Standard Algorithm Validation Suite (AESAVS) and published AES examples/test vectors.

[3] Python Software Foundation, Python 3 Documentation - tkinter, csv and standard library modules.

[4] FastAPI Documentation, REST API framework for Python.

[5] Docker Documentation, containers and Docker Desktop.

[6] Matplotlib Documentation, Python visualization library.

[7] Python Cryptographic Authority, cryptography documentation - authenticated encryption / AES-GCM utilities.

Appendix A - Principal Backend Interfaces

Endpoint	Purpose
/aes/visualize	Return instrumented AES-128 states and ciphertext.
/aes/key-expansion	Return W0-W43 derivations and 11 round keys.
/aes/avalanche	Return per-step avalanche analysis for one flipped bit.
/aes/lab/avalanche	Run a multi-trial plaintext/key avalanche experiment and return observations/statistics.

Appendix B - Final Project Workflow Summary

Dashboard -> select module -> enter plaintext/key -> send REST request -> execute AES/key/experiment logic in Docker -> return structured JSON -> render matrices, derivations, graphs or statistics -> optionally export CSV or generate a lab record.